Kestrel Home — Warranty Claim Review (Variant C)

Done By: Karthika S (rskarthika08@gmail.com)

1. Problem I Found:

Ritu Deshpande asked for a model that identifies fraudulent warranty claims before Kestrel pays them, with a target accuracy above 97%.
However, only **141 out of 11,146 decided claims (1.27%)** were fraudulent. Approving every claim would already achieve 98.7% accuracy without detecting any fraud. Therefore, accuracy alone is not a useful measure.
The investigation team can review only **40 claims per month**, and holding a genuine claim costs approximately ₹380 in customer goodwill. The main goal is to identify which 40 claims should be reviewed first.

The raw data also had several quality issues:

- **Duplicate submissions:** 12,029 rows contained only 11,348 unique claims, meaning 681 repeat submissions (5.7%).
- **Undecided claims:** 202 claims had no final investigation outcome and were excluded from training.
- **Legacy data:** 4,560 older Zoho claims were labelled as non-fraud, although some may have been undecided.
- **Serial numbers:** Approximately 30% contained inconsistent formatting.
- **Policy violations:** 27 claims worth ₹2,000 or more were paid without inspection after 1 May 2026.

I handled these issues during preprocessing before training the model.

2. What I Found Out:

The analysis showed that the fraud rate increased from **1.08% before 1 May 2026 to 2.53% afterward**, following the introduction of automatic approval for smaller claims.

The main findings were:

- New partners onboarded from October 2025 had a fraud rate of 16.1% after the policy change, compared with 0.17% for older partners.
- Small claims from newer partners had 34 fraud cases among 161 claims (21.1%).
- Only five newer outlets accounted for most of the fraud among newer partners, showing that not all new partners were suspicious.
- The highest-risk outlets included SP3160, SP3232, SP3318, SP3129 and SP3319.

Model Validation:

I trained the model on earlier claims and tested it on unseen June 2026 data.

- June claims: 720
- Actual fraudulent claims: 22
- Claims selected for review: 38

- Fraudulent claims detected: 14
- Genuine claims incorrectly flagged: 24
- Fraud detection rate: 64%
- Estimated net saving: ₹284 per reviewed claim

The model performed better than simple approaches such as reviewing the highest-value claims or prioritising all new partners.

The main finding was that **net money saved per claim reviewed is more useful than overall accuracy**.

3. How I Built the Output:

I Built the Solution in Three Stages:

1) Version 1 – Terminal:

The complete pipeline runs using:

python run_pipeline.py

It performs data cleaning, feature engineering, fraud-pattern analysis, model training, validation and prediction generation.

I used two machine-learning models:

- Logistic Regression
- Gradient Boosting

Their predictions are combined to improve fraud detection. I also used time-based validation to avoid training on future information.

2) Version 2 – CSV:

The pipeline generates structured output files, including:

- predictions.csv – Final fraud predictions for 2,252 test claims.
- scored_claims.csv – Fraud scores, risk levels and explanations.
- data_quality_report.csv – Data-cleaning results.
- fraud_pattern_analysis.csv – Fraud trends and partner patterns.
- validation_report.csv – Model validation results.

All seven submission-format checks passed.

3) Version 3 – Streamlit and FastAPI:

I created a FastAPI scoring service and a Streamlit dashboard.

- FastAPI accepts claim information and returns the fraud score, estimated fraud probability, risk level, recommended action and reasons.
- Streamlit presents the results visually, allowing employees to score individual claims and review fraud trends, financial findings and model performance.

4. Project Structure:

I divided the project into a few main parts so that each component has a clear purpose and the system is easy to maintain.

1) data/ – Input Data

Contains the original claim, partner, product and test datasets provided by Kestrel.

2) source/ – Configuration and AI Setup

Contains common application settings.

Main sub-parts:

- config.py – File paths, business rules and model settings.
- llm_client.py – Optional GPT explanation connection and usage tracking.
- prompts.py – Prompts used to generate claim explanations.

3) modules/ – Main Processing

Contains the main processing logic.

Main sub-parts:

- preprocess.py – Cleans data, removes duplicates and creates features.
- train_model.py – Trains the fraud models and performs validation.
- analyze_claim.py – Analyses fraud patterns and generates explanations.
- predict_claim.py – Predicts fraud risk for an individual claim.
- generate_predictions.py – Generates predictions for all test claims.
- validation.py – Checks data quality and prediction formats.

4) output/ – Generated Results

Stores the CSV reports, scored claims and validation results.

The model/ folder stores the trained model and its settings, while evidence/ stores charts, confusion matrices and supporting analysis.

5) app.py, api.py and run_pipeline.py – Main Application

- app.py – Starts the application and dashboard.
- api.py – Provides the fraud-scoring API.
- run_pipeline.py – Runs the complete processing pipeline.

Flow:

Clean → Feature Engineering → Train → Validate → Predict → Explain → Dashboard

6) Streamlit – Final Dashboard

The Streamlit dashboard presents the processed results in a simple format for management.

Main sections:

1. **Score a Claim** – Allows employees to enter claim details and receive a fraud score, risk level, recommended action and explanation.
2. **What the Data Says** – Displays fraud trends, high-risk outlets, monthly review queues, financial savings and model validation results.

Why I Chose It:

Instead of making Ritu examine terminal output or multiple CSV files, Streamlit provides one dashboard where she can quickly identify suspicious claims, understand fraud patterns and review model performance.

FastAPI also makes the model accessible to other applications through a scoring endpoint.

5. Limitation of the AI Analysis:

I included an optional GPT explanation feature using gpt-4o-mini. It is disabled by default and works only when an API key is configured.

The main limitations are:

- **Limited validation data:** June contained only 22 fraud cases, so the estimated financial savings are uncertain.
- **Changing fraud patterns:** A model trained only before May failed to detect the newer fraud pattern.
- **New partners:** Some partners have insufficient historical information for reliable predictions.
- **Legacy labels:** Older Zoho data may contain incorrectly labelled undecided claims.
- **Human verification:** A HIGH-risk prediction does not prove fraud and should be reviewed by an investigator.

The model also assumes a 45-day delay before investigation outcomes become available. This is a modelling assumption, not a confirmed Kestrel policy.

For production deployment, the model should be retrained regularly using newly investigated claims.

AI Token Usage and Cost:

Cost and AI Usage:

The fraud-detection model uses **scikit-learn** for local training, scoring and explanations, requiring **zero paid API calls**. GPT-4o-mini is used only for optional explanations.

Kestrel processes approximately **750 claims per month**, based on 2,252 claims over three months.

Actual API Usage Metrics:

Actual API Usage Metrics	
Metric	**Actual Usage**
Total API Requests	1
Total Tokens Used	134
Total API Cost	$0.00
Usage Period	October 4–11, 2026

Estimated GPT Cost: Each explanation uses approximately 300 input and 60 output tokens, costing around **$0.00008 per claim**. This equals $0.0032 for 40 HIGH-risk claims or $0.06 for all 750 monthly claims.

How I Reduced Token Usage:

- Used local ML models to avoid API calls for predictions.
- Kept GPT optional and disabled by default.
- Sent only essential claim details with short prompts.
- Limited responses to 150 tokens and cached previous explanations.
- Added /ai_usage to track tokens, requests and costs.

AI Tools Used: Claude for planning, coding, debugging and validation; GPT-4o-mini for optional explanations.

Overall: The core pipeline costs $0 in paid API usage, and actual testing recorded only **134 tokens with $0.00 spend**.

6. Conclusion:

- Built an end-to-end Kestrel Home warranty fraud-detection pipeline that runs locally.
- Processed **12,029 raw rows into 11,348 unique claims** after cleaning and deduplication.
- Identified that fraud increased from 1.08% to 2.53% after the May policy change.
- Trained and validated a combined Logistic Regression and Gradient Boosting model.
- Detected **14 out of 22 fraud cases** among the 38 highest-risk June claims.
- Generated predictions for **2,252 test claims**, with all seven submission-format checks passed.
- Classified claims into HIGH, MEDIUM and LOW risk, with clear explanations.
- Developed a FastAPI scoring service and a Streamlit dashboard.
- Built the system to work without a paid AI API key.

Final Recommendation:

Kestrel should prioritise the 40 highest-risk claims each month, investigate the five outlets showing unusual fraud patterns and review the 27 large claims paid without inspection.

The success of the system should be measured using **frauds detected and net money saved**, rather than accuracy alone.

- Three min Screen Recording Link:

Screen Recording - Karthika S (Kestrel Home Warranty Claim Review Link)

- Github Repo Link:

Github Repositry - Kestrel-Home-Warranty-Claim-Review-Variant-C - Karthika S

- Stream:

Kestrel Home Warranty Claim Check · Streamlit